ELEC 475 LAB 3

Sean Lawrence, Matthew Morcos

20273059, 20294076

Prof. Michael Greenspan

ELEC 475 – COMPUTER VISION WITH DEEP LEARNING

Queen’s University

10/28/2025

Table of Contents

Network Architecture

Our model, called SMNet (Simplified Custom Segmentation Model), is a lightweight semantic segmentation network built for fast and efficient inference while keeping accuracy competitive through knowledge distillation. It follows an encoder–decoder structure with skip connections, taking inspiration from U-Net but redesigned with a few key adjustments to make it leaner and more practical.

SMNet has a symmetric encoder–decoder setup with four resolution levels and skip connections at each stage. The model takes in 224×224×3 RGB images and outputs segmentation maps of size 224×224×21, which correspond to the 21 classes in the PASCAL VOC 2012 dataset (20 object categories plus background).

The main building block of SMNet is the CustomConvBlock, which combines several modern components. It starts with a convolution layer followed by batch normalization and a GELU activation. Then, a 1×1 convolution refines the features at the channel

For the decoding path, we use the SimpleUpsampler block. This module upsamples the higher-resolution features with bilinear interpolation, processes the lower-resolution features with a 1×1 convolution to reduce channel count, concatenates both, and fuses them with a 3×3 convolution followed by batch normalization and GELU/

The model begins with a stem block that processes the raw input (3 channels) into a richer feature map with 16 channels, which is downsampled by the encoder until a depth of 80 is reached. The bottleneck keeps the same depth (80→80) while refining features before upsampling begins.

Standard Training Results

With this model multiple experiments were performed to attempt to achieve the best results. First training was conducted with a base of 16 with a standard

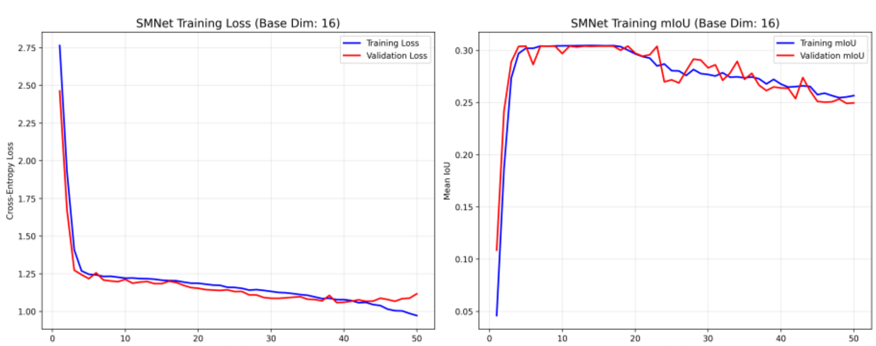

Figure 1:Loss and mIoU base 16 training plots

It was also trained with a base dimension of 32 with a higher parameter count, but this achieved similar mIoU and was much heavier.

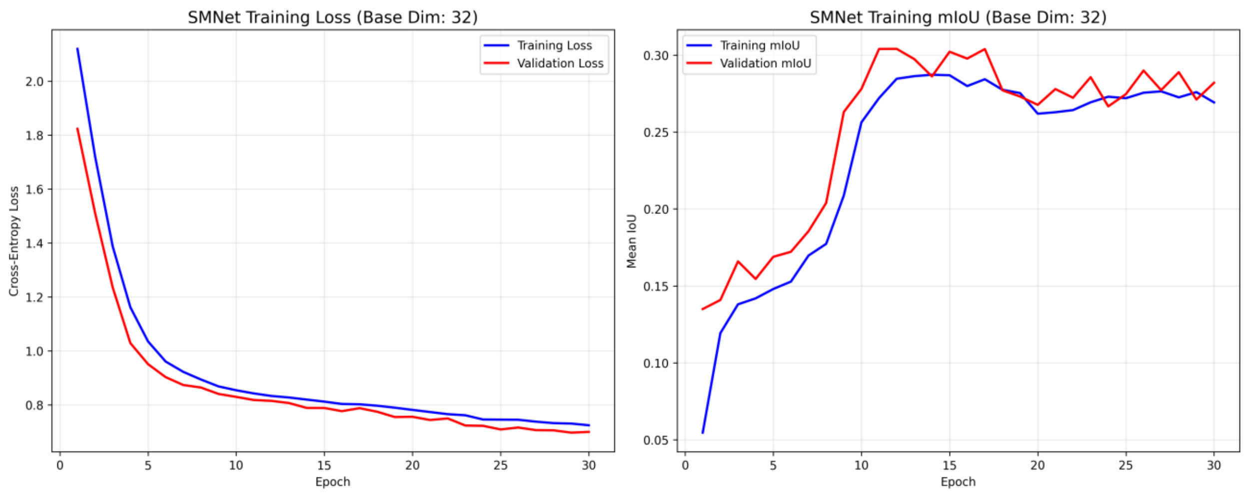

Figure 2: Loss and mIoU base 32 training plot

The 16 base is the results used going forward, in this report. The model was unable to perform correctly with these trainings despite having a higher mIoU than found in training the distilations

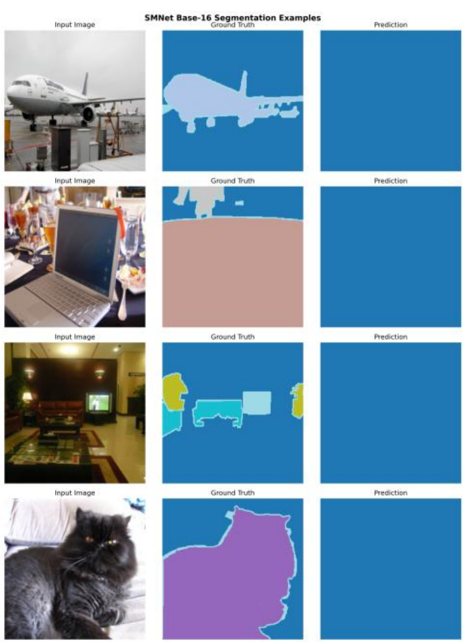

Figure 3:Base 16 Prediction results

As the model would just pick the background but no other, this did not fair any better with the 32 dimensional base.

Knowledge Distillation

Our knowledge distillation framework integrates multiple strategies to effectively transfer knowledge from a large teacher model, FCN-ResNet50, to a lightweight student model, SMNet. The approach combines response-based, feature-based, and hard-target distillation components, allowing the student to capture both the output behavior and intermediate representations of the teacher.

Overall Distillation Framework

While the distillation process was broken up into two folders, one for response, and the other for feature extraction, they both followed the same format. The initial steps were importing the database from local storage. It would create the transformations for these databases, using squeeze and long, ToTensor, and a resize functions to ensure the data will be in proper condition to test and validate with the model.

The distillation blocks backpropagate the loss functions to the student model. These components start by loading the both the student model and ResNet50. The next component is the creating the adaption layers. Given SMNets compact structure, it produces smaller channel depth being 32,48,64 channels. The teacher model of ResNet5 is producing 256,512,1024. The three adaption layers perform convolutions on the student channels to align them with the teachers. The next step would be the introduction of the standard loss function. Feature based and response base split with differing loss functions for the different goals.

Loss Functions

The first loss function used is the Focal Loss. The focal loss is used in both models. It was added as an addition to control how strongly each pixel's error contributes to the gradient. This was because in early attempts the model would attempt to decrease loss by classifying as many things as possible as background. With this if background pixels are predicted with higher accuracy, it will decrease its weight correspondingly.

The response-based distillation loss function that the standard given in the lab documentation.

$$L(x;W) = \alpha H(y,\ \sigma(z_s;T=1)) + \beta H(\sigma(z_t;T=\tau),\ \sigma(z_s;T=\tau))$$

This being is implemented with z_t being the large pre-trained model and z_s being the SMNet. The equation is broken into two parts being the hard and soft term.

The hard term (first term) is ordinary supervision from ground truth labels. It compares the student model to the pretrained using cross-entropy. In this implementation as mentioned the focal loss is used in place of this to better distribute the weights magnitude. The soft term (second term). Is what is transferring out the knowledge from teacher to student. It grabs the logic from both models and divides them by T (temperature) to get the soft probabilities. The difference is then found between the two probabilities and added to the hard term to the total loss.

The feature-based approach to distillation aligns the feature representation of the student with the cosine similarity loss of the teacher. The total loss combines these two, weighting to balance an overreliance on one or the other is focused on the representation of the models. This is done using the cosine loss.

Training Process

Both the feature and response distillation training loops would run for 70 epochs. This was determined after very lack luster mIoU numbers on smaller epoch schedules. As scene in the response mIoU training the plateau only broke around the 23 epoch and continues to climb from there. Shorter training would leave value on the table.

Each training loop would follow the same general structure as mentioned above with the models being initialized, the data being prepped and the results being put through the loss function. The next step would be the backwards pass and Optimization. An Adam optimizer was used to propagate these losses back to update the weights. After each epoch if the mIoU is greater than the last saved value it will become the new saved value, working at a checkpoint. This ensured even if training failed backups would be made.

Experimental Results

These models took many experiments to achieve the results. Attempting different ratios for the feature based and differing hyper-parameters, but the final results were below

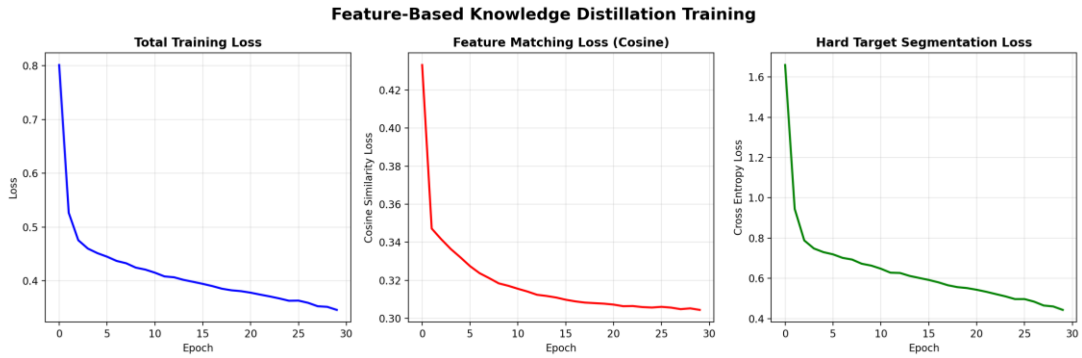

Figure 4:initial 28 Epoch training of Feature-based distillation with loss

These initial results failed due to syntax errors, and did not capture the mIoU output on each epoch, but they did save the checkpointed weights. The remaining epochs were trained Achieving an maximum mIoU of ~0.088

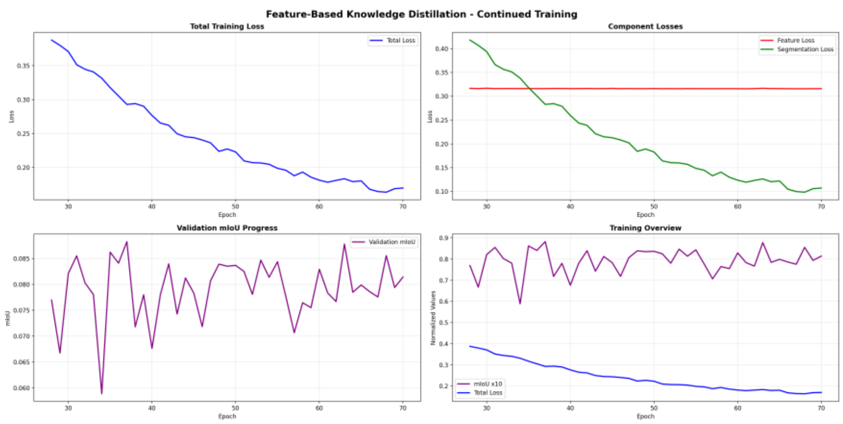

Figure 5:Feature based Remaining Epoch Training

These results were still below expected and did not produce the results hoped for as scene below.

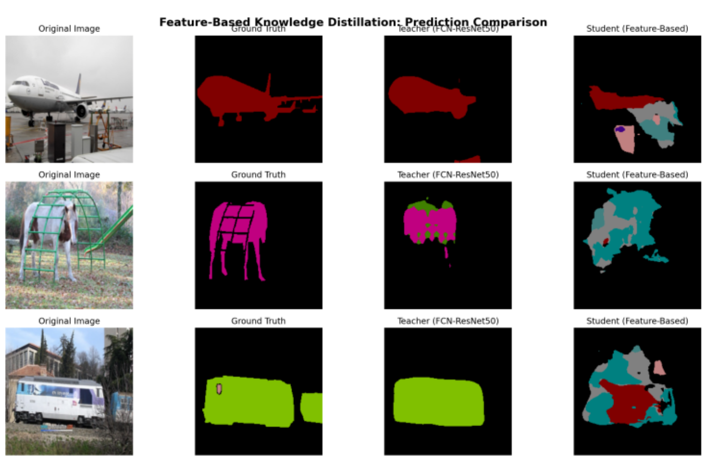

Figure 6: Example 1 of feature based distilled student model predictions

Figure 7: Example 2 of feature based distilled student model predictions

While this wasn't a full success the model did still drastically improve from the distillation as scene below with an 11.9% knowledge transfer success. This could theoretically improved by continuous training or more tweaks of the hyper parameters.

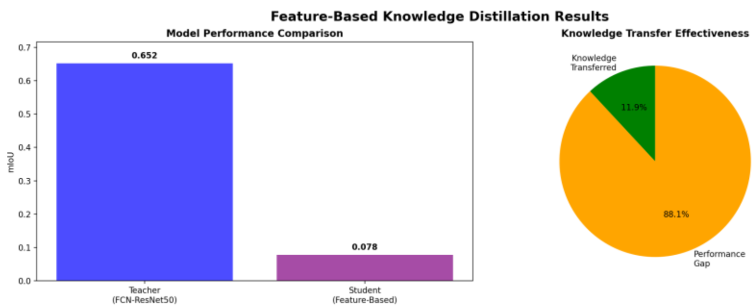

Figure 8:Graphic showing results of improvement on student model via feature based distillation

The response-based model showed extremely interesting results and varied heavily with changes. The final version was trained using 10% distillation and 90% segmentation, these extreme numbers were chosen as others initially as others would plateau at 0.0330 mIoU but would not breakout. The hope was the advanced segmentation rate would allow the model to be trained enough to breakthrough with limited distillation. This ended up working and while the breakthrough happened later than expected it, ended up achieving similar if not superior results to the feature based model.

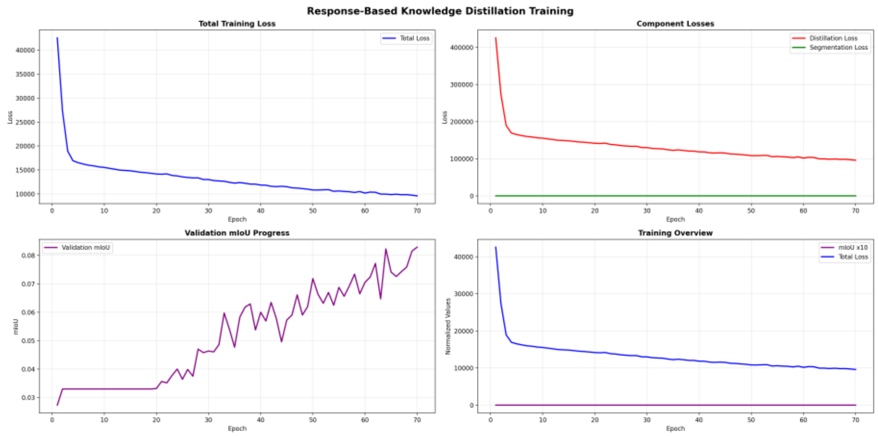

Figure 9: Response based Epoch Training

While the training did seem to progress the results ended up still being sub-par as seen below

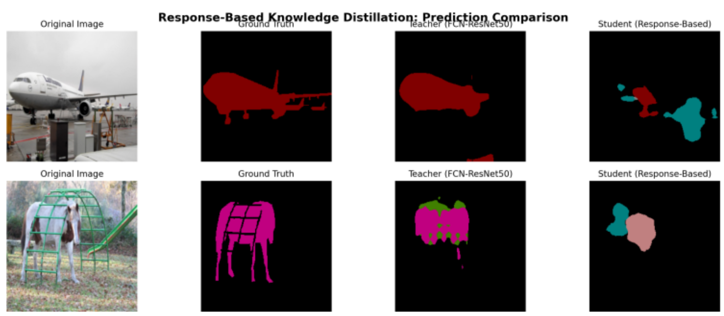

Figure 10: Example 1 of response based prediction results

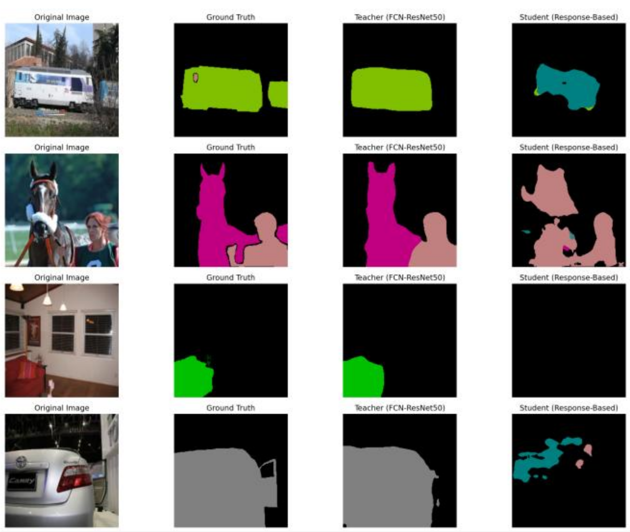

Figure 11: Example 2 of response based prediction results

There was a slightly larger transfer rate for these

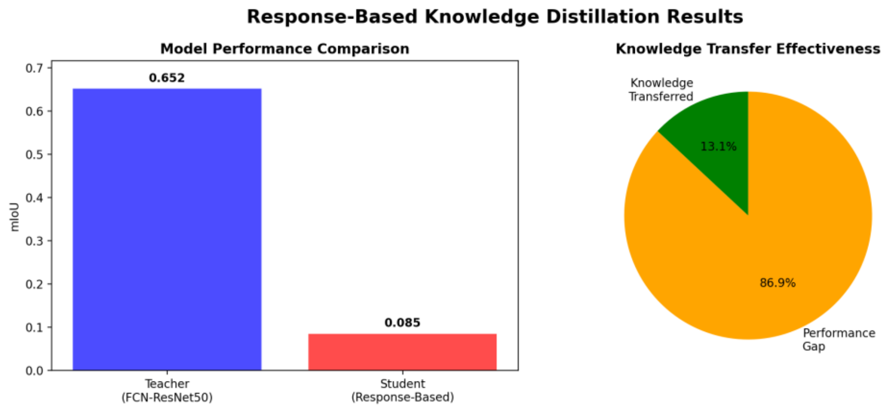

Figure 12: Graphic showing results of improvement on student model via response-based distillation

Hyperparameter Analysis and Training Data

Hyperparameters

This section describes all hyperparameters used in training our SMNet model and knowledge distillation experiments.

3.2 Baseline Training Hyperparameters

- **Optimizer:** AdamW
- **Learning Rate:** 0.001
- **Weight Decay:** 1e-4
- **Scheduler:** ReduceLROnPlateau (patience=5, factor=0.5)
- **Epochs:** 50
- **Batch Size:** 8
- **Loss Function:** CrossEntropyLoss (ignore_index=255)

3.3 Data Preprocessing Hyperparameters

- **Image Resize:** 224×224
- **Random Horizontal Flip:** p=0.5
- **ColorJitter (brightness, contrast, saturation, hue):** ±0.2 / ±0.1 ranges
- **Normalization Mean:** [0.485, 0.456, 0.406]
- **Normalization Std:** [0.229, 0.224, 0.225]

3.4 Knowledge Distillation Hyperparameters

- **Learning Rate:** 1e-4
- **Batch Size:** 4
- **Loss Weight α (Response-based):** 0.5

- **Loss Weight β (Hard target):** 0.3
- **Loss Weight γ (Feature-based):** 0.2
- **Temperature (τ):** 4.0
- **Combined Loss Formula:** $L\_total = \alpha \times L\_response \times \tau^2 + \beta \times L\_hard + \gamma \times L\_feature$

Performance and Challenges

The performance of all models was drastically under what was hoped and expected. The distillation process itself was a mild success. Both the response based and the feature based models improved over the course of training with the help of the parent/teacher model.

Knowledge Distillation	mIoU	# Parameters	Inference Speed
Teacher (FCN-ResNet50)	0.6517	35322218	17.9 ms (55.9 FPS)
Baseline SMNet (No KD)	0.304	313789	6.5 ms (153.8 FPS)
Feature-based KD	0.0776	313789	4.3 ms (234.7 FPS)
Response-based KD	0.0851	313789	3.1 ms (320.9 FPS)

Table 1:Comparing distillation mIoU values, parameters, and inference speeds

So the question remains what went wrong. While there were a variety of factors which led to this underachievement, the main one would likely be the small amount of parameters in the SMNet model constructed. The design of the model was mean to be as small, compact and optimized as possible. Unfortunately, in this effort to reduce parameters the performance clearly suffered. In future this will need to be considered.

LLM Assistance

LLMs were very helpful in this endeavor trying to properly implement the loss algorithms and structure everything properly. They did struggle heavily with any thing related to the creation of the standard model. It was important to instruct Co-Pilot not to go above and beyond as it would often attempt to make changes outside the scope of this course with things that were not relevant. This can be found multiple time with us correcting the LLM or asking for reversions. It was extremely helpful with the output of all data into figures. They made it very easy to model the loss chart and the mIoU over time. All logs can be found in the CoPilotLog.txt file inside this zip file.